

마이바티스 기초와 환경설정

오늘의 강의 학습 요약

금일 강의는 JDBC 대비 마이바티스(MyBatis)의 목적과 장점을 정리한 뒤, 마이바티스 사용을 위한 환경 구성 (라이브러리 다운로드, 빌드 패스, 문서 활용)을 진행했습니다. 이어서 DB 접속정보를 properties로 외부화하고, DTO 설계 규칙(컬럼명 일치, 기본 생성자/게터/세터 필수)을 통해 자동 매핑이 동작하는 원리를 설명했습니다.

마지막으로 마이바티스의 핵심인 SQL 매핑 흐름을 구성했습니다. configuration.xml과 mapper.xml을 작성하고 DTD 규칙에 맞춰 태그/순서를 준수했으며, SqlSessionFactory와 SqlSession을 생성하는 자바 코드까지 연결했습니다. selectOne, selectList 호출 규칙과 resultType, parameterType, #{ } 치환, 별칭(type alias) 및 DTO/Map 기반 파라미터 전달을 실습 형태로 정리했습니다.

1. 라이브러리와 프레임워크, 마이바티스 개요

라이브러리는 클래스(파일)를 가져다 쓰는 도구이고, 프레임워크는 개발을 위한 정해진 틀과 규칙까지 제공하며 마이바티스는 SQL을 메서드 호출과 연결하는 SQL 매핑 기반 도구입니다.

라이브러리는 외부에서 제공되는 코드를 “필요한 만큼 호출”해 쓰는 형태입니다. 자바에서는 주로 `jar` 로 제공되며, JDBC 드라이버처럼 외부 클래스 파일을 가져다 쓰는 방식이 대표적입니다.

프레임워크는 단순 클래스 제공을 넘어 “정해진 구조와 규칙”을 함께 제공하여, 누가 개발하더라도 비슷한 형태의 결과물이 나오도록 유도합니다. 이는 유지보수와 인수인계에서 큰 장점이 되며, 현업에서는 통일된 구조를 위해 프레임워크 기반 개발이 일반적입니다.

- 라이브러리는 개발자가 흐름을 주도하고 필요한 기능을 호출하는 방식으로 사용됩니다.
- 프레임워크는 정해진 규칙과 구조에 맞춰 개발해야 하며 전체 흐름 일부를 프레임워크가 주도합니다.
- 프레임워크를 적용하면 개발자별 구현 편차가 줄어 유지보수가 용이해집니다.
- 마이바티스는 관례적으로 프레임워크로 부르며, SQL 실행 방식을 메서드-매핑 형태로 표준화합니다.

마이바티스는 내부적으로 JDBC를 사용하지만, JDBC의 반복적인 코드를 줄이고 SQL 실행을 매핑 방식으로 자동화합니다. JDBC → MyBatis → JPA로 갈수록 개발자가 직접 처리하는 범위(SQL 작성/호출/매핑)가 줄어드는 흐름도 함께 정리되었습니다.

또한 “아이바티스(iBatis)”는 마이바티스의 초기 명칭이며, 현재는 마이바티스(MyBatis)가 표준적으로 사용됩니다.

핵심 키워드

#라이브러리

#프레임워크

#SQL 매핑

#MyBatis

#iBatis

2. 마이바티스 자료 다운로드와 빌드 패스 설정

마이바티스를 사용하려면 MyBatis 라이브러리 JAR와 DB 드라이버 JAR(MySQL 등)를 준비해 프로젝트 빌드 패스에 등록해야 합니다.

마이바티스는 외부 제공 도구이므로 반드시 라이브러리 파일을 내려받아 프로젝트에 의존성으로 추가해야 합니다. 강의에서는 MyBatis 공식 사이트(문서 및 다운로드)에서 배포 파일을 내려받고, 압축을 풀어 JAR를 확보하는 흐름을 다뤘습니다.

특히 최신 버전에서는 배포 형태(assets 구성)가 달라 JAR 포함 버전을 선택해야 하는 상황이 있었고, 예시로 3.5.17 배포 파일을 내려받아 JAR를 확보했습니다. 문서는 웹 기반 API 문서(영문/한글)와 벤더 제공 한글 PDF를 활용할 수 있으며, 외부 프레임워크는 문서의 가이드를 “정해진 대로” 따라야 안정적으로 동작한다는 점을 강조했습니다.

이클립스에서는 Build Path 에 외부 JAR 2개(MyBatis JAR, MySQL Connector JAR)를 함께 등록해야 정상 동작합니다. 한 폴더에 JAR를 모아 관리하면 재설정 시 실수를 줄일 수 있습니다.

항목	필요 이유	예시 파일	등록 위치(이클립스)
MyBatis 라이브러리	SQL 매핑 기능 제공	mybatis-3.x.x.jar	Build Path → Libraries → Classpath
DB 드라이버	JDBC 기반 DB 연결에 필요	mysql-connector-j.jar	Build Path → Libraries → Classpath

의존성 등록은 시간이 지나면 쉽게 잊히므로, 캡처/정리 등으로 재현 가능한 형태로 남기는 습관이 필요하다는 점도 함께 다뤘습니다.

핵심 키워드

#빌드 패스

#JAR

#MySQL 드라이버

#MyBatis 문서

#의존성

3. 프로퍼티즈와 DTO 설계 규칙(자동 매핑 원리)

DB 접속 4대 정보는 `properties` 파일에 키-밸류로 외부화하고, DTO는 컬럼명과 변수명 정합성 및 기본 생성자/게터/세터를 갖춰야 자동 치환, 자동 매핑이 성립합니다.

1) JDBC 4대 정보를 properties로 외부화

JDBC에서 사용하던 4가지 정보(드라이버, URL, UID, PASSWORD)는 자바 문자열로 두지 않고 외부 파일로 분리합니다. 마이바티스에서는 설정과 SQL이 XML 중심으로 구성되므로, 접속정보도 `*.properties` 로 분리하여 재컴파일 없이 수정 가능하게 만듭니다.

프로퍼티즈 파일은 키-밸류(`key=value`) 구조이며, 주석은 `#` 를 사용합니다. 특히 값 뒤 공백이 포함되면 "공백까지 값"으로 인식되어 DB 연결 실패 원인이 되므로 공백 실수를 강하게 주의해야 합니다.

- 프로퍼티즈는 키-밸류 형태로 값을 저장하고 키로 값을 조회하는 구조입니다.
- 프로퍼티즈 값의 불필요한 공백은 실제 값으로 인식되어 장애 원인이 됩니다.
- 키는 충돌 방지를 위해 `jdbc.driver` 처럼 접두어를 포함해 유니크하게 구성하는 방식이 유용합니다.
- 프로퍼티즈 파일은 `src` 하위 패키지(예: `com.config`)에 두어 클래스패스에서 접근 가능하게 구성합니다.

2) 인코딩(UTF-8) 확인과 프로퍼티즈 파일 인코딩 설정

워크스페이스 기본 인코딩은 UTF-8인 경우가 많지만, 이클립스 구버전에서는 다를 수 있어 반드시 Preferences 에서 확인해야 합니다. 또한 `.properties` 파일은 기본 인코딩이 `ISO-8859-1` 로 설정된 경우가 있어, 자바 소스(UTF-8)와의 불일치로 한글 깨짐이 발생할 수 있습니다.

구분	설정 위치	권장 값	목적
워크스페이스 인코딩	Window → Preferences → General → Workspace	UTF-8	한글 포함 텍스트 안전 처리
properties 파일 인코딩	파일 우클릭 → Properties(또는 Resource)에서 변경	UTF-8	자바/설정 파일 인코딩 정합성 유지

3) DTO는 MyBatis에서 사실상 필수이며 자동화의 핵심입니다

DTO는 레이어 간 데이터 전달뿐 아니라, 마이바티스에서 **파라미터 치환**과 **결과 매핑**을 자동화하기 위한 핵심 객체입니다. 일반적으로 테이블당 DTO 1개를 만들며, DTO의 변수명은 컬럼명과 일치시키는 것이 권장됩니다.

또한 기본 생성자와 게터/세터는 마이바티스 내부 리플렉션 기반 매핑에 의해 사용되므로 필수로 취급됩니다.

항목	JDBC에서의 성격	MyBatis에서의 성격	이유
DTO 사용	선택적일 수 있습니다.	사실상 필수입니다.	파라미터 치환과 결과 자동 매핑에 사용됩니다.
기본 생성자	선택적일 수 있습니다.	필수에 가깝습니다.	결과 매핑 시 기본 생성자로 객체를 생성합니다.
Getter/Setter	선택적일 수 있습니다.	필수에 가깝습니다.	<code>#{ }</code> 치환과 결과 매핑 시 메서드 호출이 필요합니다.
변수명-컬럼명 정합성	선택적입니다.	강력 권장입니다.	불일치 시 명시적 매핑 작업이 추가로 필요합니다.

컬럼명과 DTO 프로퍼티명이 다르면 자동으로 매핑되지 않으므로, 별도의 명시적 매핑(추후 다룰 resultMap 등)이 필요해 공수가 증가합니다.

핵심 키워드

`#properties`

`#UTF-8`

`#DTO`

`#게터/세터`

`#컬럼명 일치`

4. XML(DTD) 기반 설정과 매퍼 작성 규칙

마이바티스는 설정용 XML과 SQL 매퍼 XML을 사용하며, DTD/XSD로 정의된 규칙(태그명, 순서, 개수, 속성)을 반드시 따라야 합니다.

마이바티스는 `.java` 외에 `properties`, `xml` 파일을 함께 사용합니다. XML은 태그 기반 마크업이며, 시작/종료 태그, 중첩 구조, 속성(키-밸류), 단일 루트 태그, 빈 태그 표현을 규칙으로 갖습니다.

HTML과 XML은 모두 마크업이지만, HTML은 태그/속성이 정해져 있는 반면 XML은 태그를 사용자 정의할 수 있습니다. 다만 마이바티스에서 사용하는 XML은 “내 마음대로”가 아니라 DTD에 의해 엄격히 규칙이 정해져 있습니다.

- XML은 단일 루트 태그가 필수이며, 태그는 중첩 구조로 표현될 수 있습니다.
- 시작 태그는 속성을 가질 수 있으며 속성은 키-밸류 형태로 작성됩니다.
- 콘텐츠가 없는 태그는 `<tag />` 형태로 축약할 수 있습니다.
- 마이바티스 XML은 DTD 규칙을 따르며 정의되지 않은 태그/속성은 오류를 유발합니다.

DTD 다운로드 오류 해결(이클립스 설정)

설정 XML 상단 `DOCTYPE` 에서 DTD를 내려받지 못하면 오류가 발생합니다. 이클립스에서 외부 리소스 다운로드 옵션을 켜야 합니다.

문제	원인	해결
DTD 관련 에러 발생	외부 DTD 다운로드 차단	Preferences → XML → (Wild Web Developer 등) → Download external resources 체크

configuration.xml에서 해야 하는 핵심 작업

설정 XML에서는 DB 연결(DataSource/TransactionManager)과 매퍼 등록(mappers)을 수행하며, 프로퍼티즈 파일을 로드해 `${...}` 로 값을 치환하도록 구성합니다. 여기서 패키지 경로는 자바의 `.` 대신 `/` 를 사용한다는 점이 중요합니다.

구분	태그/속성	역할	주의점
properties 등록	<code><properties resource="..." /></code>	외부 키- 밸류 파일 로드	<code>com/config</code> 처럼 슬래시 경로를 사용합니다.
environments	<code><environments default="..."></code>	실행 환경 묶음	DTD가 요구하는 구조입니다.
transactionManager	<code><transactionManager type="JDBC" /></code>	트랜 잭션 처리 방식	트랜잭션 개념 지 수합니다.
dataSource	<code><dataSource type="POOLED"></code> 등	DB 연결 정보	내부적으로 JDBC 연결을 관리합니다.
mapper 등록	<code><mappers><mapper resource="..." /></mappers></code>	SQL 매퍼 XML 등록	등록 누락 시 SQL 실행이 불가능합니다.

mapper.xml에서 SQL 정의 규칙

매퍼 XML은 SQL을 태그로 저장합니다. CRUD에 해당하는 `select`, `insert`, `update`, `delete` 태그를 사용하며, 각 태그의 `id` 는 필수이고 유일해야 합니다. SQL 문장은 세미콜론 `;` 을 붙이지 않습니다.

또한 JDBC의 `?` (물음표) 바인딩 대신 마이바티스는 `#{} (샵 블록)` 을 사용합니다.

요소	의미	JDBC 대비
<code>id</code>	SQL 식별자이며 호출 시 첫 번째 인자로 사용됩니다.	메서드-매핑 연결점입니다.
<code>resultType</code>	조회 결과(레코드)를 담은 타입입니다.	결과를 DTO로 자동 매핑합니다.
<code>parameterType</code>	<code>#{ }</code> 에 채워질 파라미터의 타입입니다.	물음표에 넣던 값을 자동 치환합니다.
<code>#{name}</code>	파라미터 바인딩 표현식입니다.	<code>?</code> 를 대체합니다.
<code>;</code>	사용하지 않습니다.	매퍼 SQL에서는 세미콜론을 제외합니다.

핵심 키워드

`#XML`

`#DTD`

`#configuration.xml`

`#mapper.xml`

`##{ }`

5. SqlSessionFactory/SqlSession 생성과 SQL 호출(selectOne/selectList)

configuration.xml을 자바 코드에서 읽어 SqlSessionFactory를 만들고, SqlSession을 열어 ID 기반으로 SQL을 실행하며 단일행은 selectOne, 다중행은 selectList로 처리합니다.

마이바티스에서 실제 실행 흐름은 다음으로 정리됩니다. 설정과 매핑은 XML에 있고, 자바는 "설정 파일을 읽고 세션을 열어 실행하는 역할"을 담당합니다.

- SqlSessionFactory 는 설정을 기반으로 SqlSession 을 생성하는 공장 역할입니다.
- SqlSession 은 JDBC의 Connection/Statement 역할을 합친 것처럼 DB 연동과 실행 API를 제공합니다.
- 세션을 사용한 뒤에는 반드시 close() 로 자원을 해제해야 합니다.
- JDBC 대비 예외 처리 코드가 줄고 설정/SQL의 외부화로 유지보수가 쉬워집니다.

SqlSessionFactory 구성 코드의 배치(Static Block)

문서의 예제 코드는 실행문이므로 클래스 바깥에 그대로 둘 수 없고, 생성자/메서드/정적 블록 등 실행 가능한 블록 내부에 배치해야 합니다. 강의에서는 객체 생성 없이 초기화하기 위해 static 블록을 선택했습니다.

또한 static 블록에서 접근해야 하므로 SqlSessionFactory 필드도 static으로 선언해야 하며, getSession() 또한 객체 생성 없이 호출하려면 static 메서드로 구성합니다.

```
public class MySqlSessionFactory {  
    private static SqlSessionFactory sqlSessionFactory;  
  
    static {  
        // Resources.getResourceAsStream(...) 등으로 configuration.xml을 읽고 factory를 초기화합니다.  
    }  
  
    public static SqlSession getSession() {  
        return sqlSessionFactory.openSession();  
    }  
}
```

단일행 조회: selectOne

단일 레코드 조회는 selectOne 을 사용합니다. 첫 번째 인자는 매핑 태그의 id 이며, 두 번째 인자는 #{ } 에 들어갈 파라미터 값입니다. 파라미터가 없으면 selectOne(String) 형태를 사용합니다.

구분	호출 형태	의미
파라미터 없음	<code>selectOne(id)</code>	<code>#{ }</code> 가 없는 SQL 실행입니다.
파라미터 있음	<code>selectOne(id, param)</code>	<code>#{ }</code> 가 있는 SQL 실행이며 <code>param</code> 이 바인딩에 사용됩니다.

다중행 조회: selectList

여러 레코드 반환은 `selectList` 를 사용하며, 반환 타입은 `List<T>` 형태로 DTO 리스트가 됩니다. 파라미터가 있으면 두 번째 인자에 전달합니다.

구분	호출 형태	반환
파라미터 없음	<code>selectList(id)</code>	<code>List<resultType></code>
파라미터 있음	<code>selectList(id, param)</code>	<code>List<resultType></code>

파라미터 전달 방식: DTO vs Map

`parameterType` 이 DTO이면 `#{deptNo}` 같은 값은 DTO의 `getDeptNo()` 호출 결과로 바인딩됩니다.
`parameterType` 이 Map이면 `#{x}` 는 Map의 키 "x" 에 해당하는 값으로 바인딩됩니다.

<code>parameterType</code>	<code>#{key}</code> 해석 방식	예시
DTO	<code>getKey()</code> 를 호출한 값	<code>#{deptNo}</code> → <code>getDeptNo()</code>
Map	Map에서 "key" 로 조회한 값	<code>#{x}</code> → <code>map.get("x")</code>

resultType/parameterType 별칭(Type Alias)

문서의 type alias 테이블에 정의된 별칭을 사용하면 `java.lang.Integer` , `java.util.HashMap` 같은 풀네임 대신 `int` , `hashmap` 등 소문자 별칭으로 표기할 수 있습니다. 이는 매퍼 XML 가독성과 작성 편의를 높입니다.

별칭	실제 타입
int	java.lang.Integer
hashmap	java.util.HashMap
list	java.util.List
arraylist	java.util.ArrayList

핵심 키워드

#SessionFactory

#Session

#selectOne

#selectList

#type alias